

Solicitação de acesso técnico à API da WebLocação

De: Optus Agent — plataforma ALFAIA **Para:** Equipe técnica WebLocação **Assunto:** Liberação de acesso de integração (leitura de produtos/agenda e escrita de agendamento) **Data:** 08/08/2026 **Referente a:** demonstração e alinhamento de escopo de 07/08/2026

1. Contexto

A Optus Agent está desenvolvendo o **ALFAIA**, uma plataforma de atendimento via WhatsApp e CRM conversacional para locadoras de trajes. O ALFAIA é um produto próprio da Optus Agent, independente da WebLocação — a WebLocação atua como canal de indicação para os nossos clientes finais (locadoras), não como sócia da plataforma. Essa fronteira já foi alinhada na reunião de 07/08/2026 e orienta tudo o que pedimos abaixo: **o ALFAIA nunca escreve em contrato, pedido, financeiro, parcela, nota fiscal ou estoque, nunca mantém catálogo próprio, e nunca substitui nenhuma tela do sistema de vocês.** Nosso produto é a camada de conversa e relacionamento ao redor do ERP — não uma extensão dele.

Para isso, o motor de atendimento do ALFAIA precisa consultar produtos e agenda em tempo real, e criar agendamentos, diretamente na API de vocês. Este documento formaliza o pedido de acesso técnico necessário para isso.

2. O que estamos pedindo

Recurso	Nível de acesso solicitado	Uso
Produtos e status dos produtos	Leitura	O atendente de IA consulta disponibilidade em tempo real durante a conversa com o lead — nunca mantemos catálogo espelhado
Agenda — horários e vagas	Leitura e escrita	Consultamos vagas livres e criamos o agendamento quando o lead confirma, diretamente na agenda de vocês

Não pedimos acesso a nenhum outro módulo (financeiro, contratos, estoque, consignação, cadastro completo de cliente) — o que está fora dessa tabela está fora do nosso escopo por princípio de produto, não por limitação técnica.

3. Endpoints necessários e finalidade de cada um

Abaixo está o contrato que assumimos como ponto de partida, com base no que foi apresentado na reunião. **Pedimos que confirmem ou corrijam cada um** — os formatos exatos de request/response, paginação e autenticação ficam a critério de vocês; o que descrevemos é o que precisamos que o endpoint faça, não uma exigência de formato.

Endpoint	Método	Finalidade de negócio
<code>/produtos</code>	GET	Buscar peças disponíveis por evento, categoria, tamanho, cor, estilo e período — é o que a IA usa quando o lead descreve o que procura
<code>/produtos/{id}</code>	GET	Detalhar uma peça específica, incluindo os períodos em que está disponível
<code>/agenda/slots</code>	GET	Consultar vagas livres por tipo de compromisso (ex.: prova) num intervalo de datas
<code>/agenda</code>	POST	Criar um agendamento quando o lead confirma horário — precisamos escrever aqui
<code>/agenda</code>	GET	Ler os agendamentos de um período, para exibirmos numa tela de agenda somente-leitura no nosso portal

Parâmetros de filtro que esperamos poder usar em `/produtos` e `/agenda/slots`: intervalo de datas, categoria, tamanho, cor, estilo e um campo de busca livre — mas nos adaptamos ao que já existir no lado de vocês.

4. Perguntas técnicas que precisamos que respondam

Estas são as três lacunas que identificamos internamente e que travam nosso desenvolvimento contra a API real (hoje trabalhamos com um mock que assume as respostas mais prováveis, só para não parar o desenvolvimento):

- Contrato exato dos endpoints** — formato de request/response, paginação, rate limit e o método de autenticação que vocês usam para integrações de terceiros.
- Modelo de vagas da agenda** — os horários de `/agenda/slots` são por vaga individual ou por horário fixo com múltiplos atendimentos simultâneos? Existe capacidade simultânea por horário?
- Cadastro obrigatório para agendar** — ao criar um agendamento via `POST /agenda`, o cliente final precisa já existir cadastrado no ERP de vocês, ou o endpoint aceita nome + telefone diretamente (já que, do nosso lado, o lead pode nunca ter sido cadastrado manualmente na loja)?

5. Como precisamos que o acesso seja liberado

Aqui está o ponto que mais precisamos alinhar com vocês: **qual é o processo de vocês para conceder acesso de API a uma integração de terceiro?**

Do nosso lado, a arquitetura já está pronta para o modelo mais comum desse tipo de integração — **uma chave de API por cliente/loja**, que guardamos de forma criptografada, nunca exposta no código nem ao navegador do usuário, associada internamente a cada loja que atendemos. Se esse for também o processo de vocês (uma chave por loja), funciona perfeitamente sem ajuste nenhum do nosso lado.

Se o processo de vocês for diferente — por exemplo, uma única chave de parceiro/reseller que nós gerenciamos e reaplicamos por loja, ou um fluxo de aprovação/OAuth por loja — nos avisem qual é, para adaptarmos a forma como armazenamos e rotacionamos essas credenciais internamente. Não temos preferência técnica forte aqui; só precisamos saber qual processo seguir.

6. Fase atual vs. visão de produto (importante para o desenho do acesso)

Para sermos transparentes sobre o momento do produto:

Agora (piloto): vamos integrar com a API de vocês para **um único cliente específico** (a loja que faremos o piloto). Nesta fase inicial, vamos inserir uma única credencial e uma única configuração de integração, apontando para essa loja.

Em breve (visão de produto): o ALFAIA é desenhado desde a primeira linha de código como uma **plataforma multi-tenant (SaaS)** — cada loja cliente é um tenant isolado, com seus próprios dados, sua própria credencial de canal e sua própria credencial de integração com o ERP. O piloto de um cliente único é o primeiro tenant dessa arquitetura, não uma versão à parte que será refeita depois.

Por que isso importa para vocês agora: pedimos que, ao definir o processo de liberação de acesso (item 5), já considerem que, no médio prazo, vamos precisar repetir esse mesmo processo — de forma previsível e, se possível, self-service ou semi-automatizada — para cada nova loja que se tornar cliente da Optus Agent através da parceria com vocês. Não precisamos que isso já esteja pronto hoje; precisamos apenas saber se o processo de vocês **escala** para múltiplas lojas sem exigir uma negociação manual a cada uma, para planejarmos com realismo o nosso lado da integração.

7. Nossos limites técnicos e segurança

Para que vocês tenham visibilidade de como tratamos o acesso que estão nos concedendo:

Prática	Como implementamos
Armazenamento de credenciais	Chave de API por loja, guardada em coluna criptografada no banco — nunca em código-fonte, nunca exposta ao navegador do usuário final
Timeout de chamada	8 segundos por chamada. Se a API de vocês não responder nesse prazo, nunca travamos a conversa do lead — informamos que vamos confirmar e seguimos por outro caminho
Retentativas	Máximo de 2 tentativas em caso de erro 5xx, com backoff. Erros 4xx não são retentados (assumimos que é um erro nosso de requisição, não instabilidade de vocês)
Auditoria	Toda chamada feita à API de vocês é logada internamente (rota, status, latência), para conseguirmos investigar qualquer divergência sem precisar acionar vocês a cada dúvida
Escrita idempotente	A criação de agendamento (<code>POST /agenda</code>) é feita de forma idempotente do nosso lado — se por qualquer motivo reenviarmos a mesma solicitação, ela não duplica o agendamento
Dado não inventado	Nosso motor de IA nunca informa disponibilidade, valor ou horário que não tenha vindo de uma resposta real da sua API na conversa em curso — é um princípio de produto, não só uma prática técnica

Sobre a rede de onde as chamadas vão partir ("túneis")

Este é um ponto que preferimos deixar explícito, porque é honesto e pode ser relevante se vocês fizerem controle de acesso por IP:

- **Fase atual (desenvolvimento e testes):** nossa aplicação roda em **Cloud Run (Google Cloud)**. Nessa fase, o IP de saída das nossas chamadas **não é fixo** por padrão — é dinâmico e compartilhado com outros serviços do Google Cloud. Se o acesso de vocês depende de **autenticação por chave/token via HTTPS**, isso não representa nenhum problema. Se, por outro lado, vocês exigem **allowlist de IP de origem**, precisamos combinar isso com antecedência, porque exigiria configurarmos um IP de saída fixo (via NAT dedicado) especificamente para essa integração — é possível, mas não é o padrão que já temos rodando hoje.
- **Fase de produção (pós-venda):** a aplicação migra para uma VPS com IP fixo e dedicado. Nessa fase, allowlist de IP por parte de vocês, se exigirem, é trivial de atender.
- Todas as chamadas, em qualquer fase, são feitas via **HTTPS** — não expomos nem aceitamos tráfego não criptografado com a sua API.
- Não expomos nenhum endpoint nosso para a WebLocação chamar de volta nesta integração — o fluxo é sempre **nós chamando vocês** (leitura de produtos/agenda, escrita de agendamento). Se em algum momento futuro isso mudar (por exemplo, um webhook de vocês nos notificando de uma alteração feita

direto no ERP), tratamos isso como uma extensão de escopo à parte, a ser desenhada e comunicada separadamente.

8. Próximos passos propostos

1. Vocês confirmam (ou corrigem) o contrato de endpoints da seção 3.
2. Vocês respondem as três perguntas técnicas da seção 4.
3. Vocês nos informam o processo de liberação de acesso (seção 5) e nos indicam se ele é repetível para múltiplas lojas (seção 6).
4. Emitem a credencial de teste para a loja piloto.
5. Fazemos um teste de integração ponta a ponta em ambiente controlado antes de qualquer chamada em produção.

Ficamos à disposição para uma call técnica, se for mais rápido do que trocar isso por escrito.

Documento preparado por Pax (@po) a partir de [PRD-ALFAIA-v2.md](#) §1.1, §6.2–§6.3, §7 e §29 (Q1–Q3) e da estratégia de deploy do §24 (emenda de 08/08/2026). Nenhuma informação de contrato de API foi inventada — o que está descrito como "assumido" ou "esperado" é explicitamente marcado como tal, para confirmação da WebLocação.